

DevOps Learning Guide

Muhammad Arham's Personal Handbook

Git | Docker | Node.js | GitHub Flow

From Zero to Deploying with Confidence

GitHub



VS Code



Docker



Ship! Ø=Þ€

Based on your actual project: demo_nodejs_webpage | Node.js + Express + Docker | Generated 2026-04-26

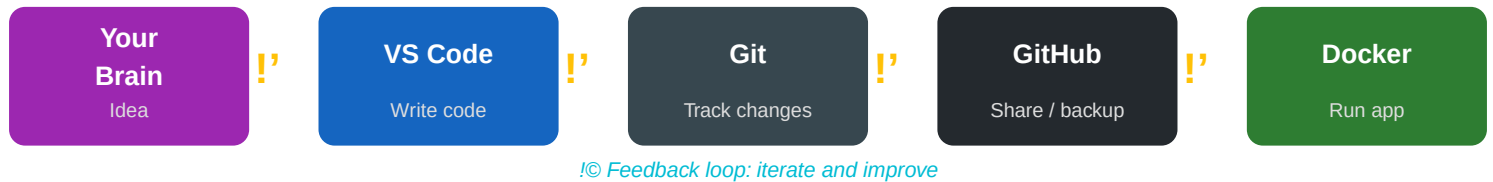
Table of Contents

1	The Big Picture — Your Workflow	3
2	Git Fundamentals & Daily Commands	5
3	Branching Strategies	9
4	Docker: Images vs Containers	12
5	Your Dockerfile — Line by Line	15
6	Common Errors & Fixes	18
7	Cheat Sheets	22

Section 1 — The Big Picture: Your Workflow

Before diving into commands, understand WHY each tool exists and how they connect. Here is your complete daily workflow as a DevOps learner:

Your Daily Workflow



Step-by-Step: What You Do Each Day

Step	Tool	What You Do	Result
1	VS Code	Open project, edit app.js or HTML files	New code written
2	Terminal	<code>git add . && git commit -m 'message'</code>	Change saved to history
3	Terminal	<code>git push origin main</code>	Code backed up on GitHub
4	Terminal	<code>docker build -t myapp .</code>	Docker image created
5	Terminal	<code>docker run -p 3000:8081 myapp</code>	Container running
6	Browser	Visit <code>http://localhost:3000</code>	You see your webpage!

Your Project File Structure

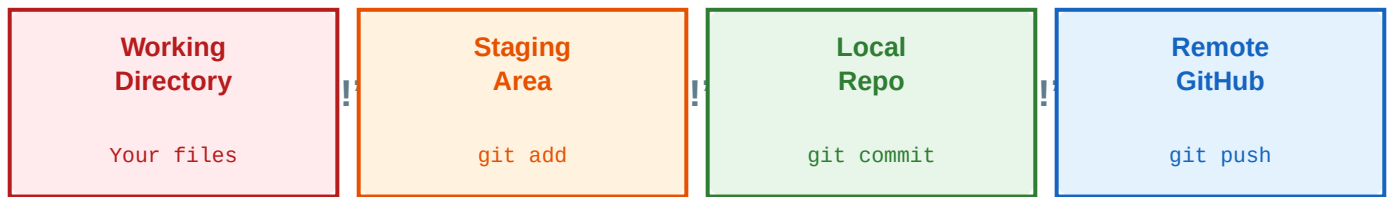
```
demo_nodejs_webpage/      !• Your project root
  % % % app.js             !• Main Express server
  % % % package.json       !• Dependencies & scripts
  % % % Dockerfile         !• Docker build recipe
  % % % index.html         !• Homepage
  % % % about.html        !• About page
  % % % routes/           !• Express route handlers
    % % % tasks.js
  % % % data/             !• Data layer
    % % % task.js
```

& p **IMPORTANT** — Port Mismatch in YOUR project: app.js listens on port 8081, but your Dockerfile says EXPOSE 3000. This means you must run: `docker run -p 3000:8081 myapp` (left=what browser uses, right=what app uses inside container). This is covered in detail in the Common Errors section.

Section 2 — Git Fundamentals & Daily Commands

Git is a version control system — it keeps a complete history of every change you make, so you can always go back, compare, or share your work.

The Git Mental Model — 4 Zones



Ø=Ü; Files live in your Working Directory. You stage selected changes, commit them to local history, then push to share on GitHub.

Daily Git Commands Reference

Command	What It Does	Example
<code>git init</code>	Create a new Git repo	<code>git init</code>
<code>git status</code>	See what's changed / staged	<code>git status</code>
<code>git add <file></code>	Stage a specific file	<code>git add app.js</code>
<code>git add .</code>	Stage ALL changed files	<code>git add .</code>
<code>git commit -m</code>	Save staged changes with message	<code>git commit -m "add route"</code>
<code>git log --oneline</code>	See compact commit history	<code>git log --oneline</code>
<code>git diff</code>	See unstaged changes	<code>git diff app.js</code>
<code>git push origin main</code>	Upload commits to GitHub	<code>git push origin main</code>
<code>git pull origin main</code>	Download latest from GitHub	<code>git pull origin main</code>
<code>git clone <url></code>	Download a repo from GitHub	<code>git clone https://...</code>
<code>git branch</code>	List all branches	<code>git branch</code>
<code>git checkout -b</code>	Create & switch to new branch	<code>git checkout -b feature/x</code>
<code>git merge <branch></code>	Merge a branch into current	<code>git merge feature/x</code>
<code>git stash</code>	Temporarily save uncommitted work	<code>git stash</code>
<code>git stash pop</code>	Restore stashed work	<code>git stash pop</code>

Good vs Bad Commit Messages

BAD — avoid these

```
git commit -m "fix"
git commit -m "changes"
git commit -m "asdf"
git commit -m "wip"
git commit -m "stuff"
```

GOOD — use these

```
git commit -m "fix port mismatch in Dockerfile"
git commit -m "add /about route to app.js"
git commit -m "update package.json description"
git commit -m "work in progress: adding task routes"
git commit -m "remove unused console.log in app.js"
```


Ø=Ü; Rule: A good commit message completes the sentence "If applied, this commit will..."

Real Workflow Using YOUR Project

```
# You just edited app.js to add a new route

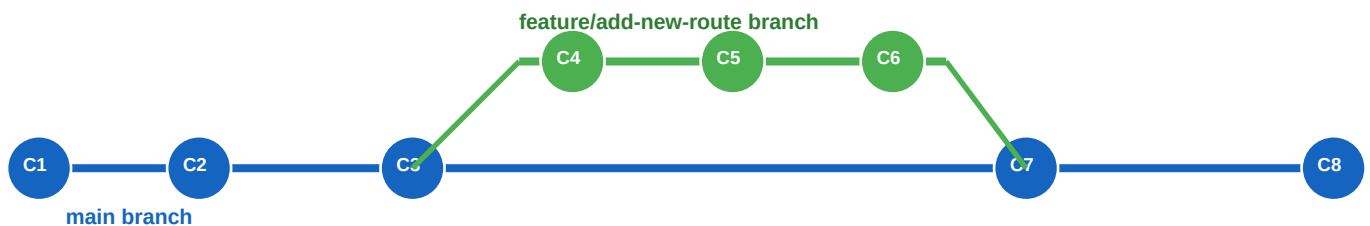
git status                # see: modified app.js
git diff app.js           # review your changes
git add app.js            # stage just that file
git commit -m "add /contact route to Express app"
git push origin main      # send to GitHub

# Later, pull changes from another machine
git pull origin main
```


Section 3 — Branching Strategies

A branch is an independent copy of your code where you can make changes without affecting the main, working version. Think of it like a parallel universe for your code.

What a Branch Looks Like



Ø=Ü; The main branch always stays working. You create a feature branch, do your work there, then merge it back when done.

GitHub Flow (Recommended for Beginners)

1	Create branch	<code>git checkout -b feature/new-route</code>
2	Make changes	Edit <code>app.js</code> , add your new feature
3	Commit often	<code>git add . && git commit -m "add route"</code>
4	Push branch	<code>git push origin feature/new-route</code>
5	Open PR	On GitHub: compare & pull request
6	Merge & delete	Merge PR, delete branch on GitHub

GitHub Flow vs Git Flow — Side-by-Side

GitHub Flow (SIMPLE — use this)	Git Flow (complex — learn later)
2 key branches: main + feature/*	5 branch types: main, develop, feature, release, hotfix
Deploy directly from main	Deploy from release branches only
Great for: small teams, web apps	Great for: large teams, versioned software
YOUR demo project: USE THIS '	Learn this when working in larger teams

Branch Naming Conventions

Branch Type	Pattern	Example for YOUR project
Feature	feature/<description>	feature/add-contact-page
Bug fix	fix/<description>	fix/port-mismatch-dockerfile
Hotfix	hotfix/<description>	hotfix/app-crash-on-startup
Docs	docs/<description>	docs/add-readme
Refactor	refactor/<description>	refactor/split-routes

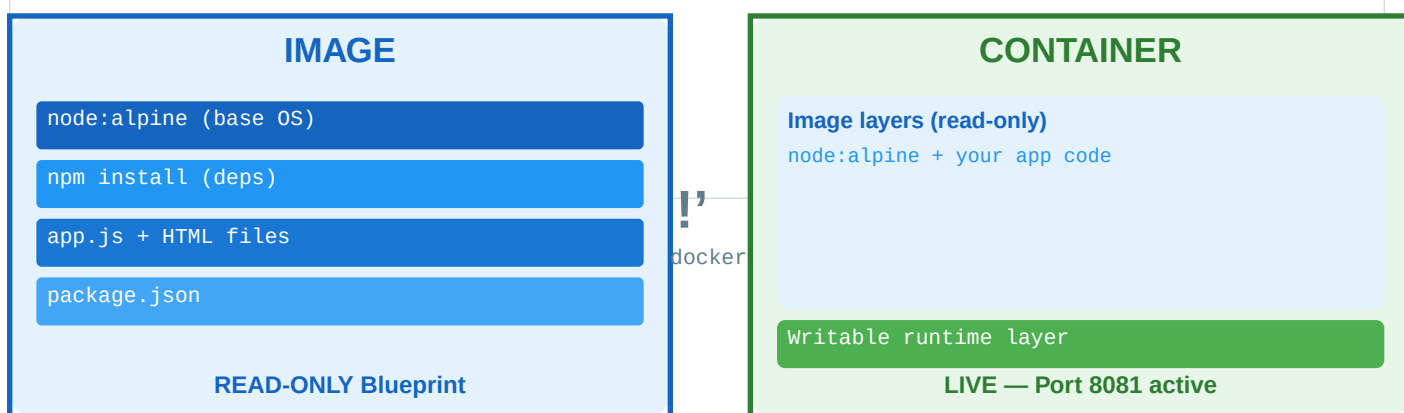
Section 4 — Docker: Images vs Containers

Docker solves the classic problem: "It works on my machine!" It packages your app with EVERYTHING it needs — Node.js, dependencies, config — into a single unit that runs identically everywhere.

The Analogy That Makes Docker Click

Real World	Docker Equivalent	In Your Project
Cookie recipe	Dockerfile	Your Dockerfile file
Cookie cutter	Image	node:alpine + your app code
Actual cookie	Container	Running instance of your app
Multiple cookies	Multiple containers	Scale to handle more users

Image vs Container — Visual Comparison



Ø=Ü; Key difference: An IMAGE is static (a snapshot). A CONTAINER is a live, running instance of that image. You can run many containers from one image.

Essential Docker Commands

Command	What It Does	Example
<code>docker build -t <name> .</code>	Build image from Dockerfile	<code>docker build -t myapp .</code>
<code>docker images</code>	List all images	<code>docker images</code>
<code>docker run -p <host>:<cont> </code>	Start container from image	<code>docker run -p 3000:8081 myapp</code>
<code>docker ps</code>	List running containers	<code>docker ps</code>
<code>docker ps -a</code>	List ALL containers	<code>docker ps -a</code>
<code>docker stop <id></code>	Stop a running container	<code>docker stop abc123</code>
<code>docker rm <id></code>	Delete a stopped container	<code>docker rm abc123</code>
<code>docker rmi <name></code>	Delete an image	<code>docker rmi myapp</code>
<code>docker logs <id></code>	View container output/logs	<code>docker logs abc123</code>
<code>docker exec -it <id> sh</code>	Open shell inside container	<code>docker exec -it abc123 sh</code>

Port mapping (-p host:container): LEFT number = browser port. RIGHT = app port inside container. For YOUR project: -p 3000:8081 !' visit localhost:3000 !' hits app on 8081.

Section 5 — Your Dockerfile, Line by Line

A Dockerfile is a recipe — a script Docker follows to build your image. Here is YOUR actual Dockerfile from `demo_nodejs_webpage/`, explained line by line:

BASE IMAGE

`FROM node:alpine`

! Start from an official Node.js image built on Alpine Linux (a tiny, secure Linux distro). This gives you Node.js and npm pre-installed. Alpine keeps the image small (~120MB vs ~900MB for full Ubuntu).

WORKING DIR

`WORKDIR /app`

! Set /app as the working directory inside the container. All subsequent commands run from here. If /app doesn't exist, Docker creates it automatically.

COPY DEPS FIRST

`COPY package*.json ./`

! Copy only package.json BEFORE copying all code. Smart trick: Docker caches this layer. If you only change app.js, npm install won't re-run — saving minutes of build time!

INSTALL DEPS

`RUN npm install`

! Run npm install to download all dependencies (like Express). These get installed into /app/node_modules inside the image.

COPY ALL CODE

`COPY . .`

! Copy everything from your local project folder into /app in the image. This includes app.js, index.html, about.html, routes/, data/, etc.

DOCUMENT PORT &

`EXPOSE 3000`

! Tell Docker (and humans) that the container intends to use port 3000. NOTE: Your app.js actually listens on 8081 — this is a mismatch! EXPOSE is just documentation; you fix it at docker run with -p 3000:8081.

START CMD

`CMD ["npm", "start"]`

! The default command when the container starts. Runs "npm start" which executes "node app.js" (as defined in package.json). Use array form ["cmd", "arg"] for better signal handling.

Current Dockerfile vs Improved Version

Current (has issues)

```
# YOUR CURRENT Dockerfile

FROM node:alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Improved version

```
# IMPROVED Dockerfile

FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 8081 # match app.js!
CMD ["node", "app.js"]
```

- node:18-alpine pins a specific Node version — builds stay reproducible.
- npm ci is faster and stricter, ideal for containers and CI pipelines.
- --only=production skips devDependencies — smaller final image.
- EXPOSE 8081 matches what app.js actually uses.
- node app.js directly is slightly more efficient than npm start in containers.

Section 6 — Common Errors & Fixes

Every developer hits these. Here are the most common errors in YOUR project workflow, with exact fixes:

ERR-01 Port Mismatch — YOUR project has this right now!

Error message:

```
docker run -p 3000:3000 myapp
# App runs but browser shows 'This site can't be reached'
```

Cause:

app.js listens on 8081, but you mapped -p 3000:3000 (container side wrong).

Fix:

```
docker run -p 3000:8081 myapp
# OR fix Dockerfile: change EXPOSE 3000 to EXPOSE 8081
```

🔗 Why it works: The RIGHT side of -p must match the port your app ACTUALLY uses inside the container.

ERR-02 Git Push Rejected

Error message:

```
! [rejected] main -> main (non-fast-forward)
error: failed to push some refs
```

Cause:

Someone else (or you on another machine) pushed commits you don't have locally.

Fix:

```
git pull origin main --rebase
git push origin main
```

🔗 Why it works: Pull first to merge their work, then push your combined history.

ERR-03 Cannot find module 'express'

Error message:

```
Error: Cannot find module 'express'
    at Function.Module._resolveFilename
```

Cause:

Dependencies not installed. node_modules folder is missing.

Fix:

```
npm install
docker build -t myapp . # rebuild image
```

🔗 Why it works: node_modules is in .gitignore (correctly). Always run npm install after cloning.

ERR-04 Merge Conflict

Error message:

```
CONFLICT (content): Merge conflict in app.js
Automatic merge failed; fix conflicts and then commit
```

Cause:

Two branches changed the same lines. Git doesn't know which to keep.

Fix:

```
# Open app.js in VS Code, look for <<<<<< markers
# Keep the version you want, delete the markers
git add app.js
git commit -m "resolve merge conflict in app.js"
```

Ø=Ü; Why it works: Conflicts are normal — Git is asking YOU to make the decision it can't.

ERR-05 Docker: address already in use

Error message:

```
Error: bind: address already in use
... port 3000 already allocated
```

Cause:

Another container (or process) is already using port 3000 on your machine.

Fix:

```
docker ps          # find running containers
docker stop <container_id> # stop the conflicting one
# OR use a different host port:
docker run -p 3001:8081 myapp
```

Ø=Ü; Why it works: Each port on your machine can only be used by one process at a time.

ERR-06 Git: nothing to commit, working tree clean

Error message:

```
nothing to commit, working tree clean
```

Cause:

Not actually an error — means you have no unsaved changes.

Fix:

```
git status      # check what's happening
git log --oneline # verify your last commit is there
```

Ø=Ü; Why it works: This message means everything is saved. Check git log to confirm.

ERR-07 Docker build: COPY failed — file not found

Error message:

```
COPY failed: file not found in build context  
or excluded by .dockerignore
```

Cause:

The file you're trying to COPY doesn't exist, or is in .dockerignore.

Fix:

```
ls -la          # verify the file exists in your project  
cat .dockerignore # check if the file is excluded  
docker build -t myapp . # the dot = current folder!
```

🔗; Why it works: Docker builds from the 'context' (folder). Always run docker build from project root.

Section 7 — Cheat Sheets

GIT CHEAT SHEET

Category	Command	What It Does
SETUP	git config --global user.name "Muhammad Arham"	Set your name
	git config --global user.email "your@email.com"	Set your email
START	git init	New local repo
	git clone <url>	Clone from GitHub
SNAPSHOT	git status	See changes
	git add <file>	Stage a file
	git add .	Stage all
	git commit -m "msg"	Save snapshot
HISTORY	git log --oneline	Compact history
	git diff	Unstaged changes
	git diff --staged	Staged changes
BRANCHES	git branch	List branches
	git checkout -b <name>	New branch
	git checkout main	Switch to main
	git merge <branch>	Merge branch
REMOTE	git push origin main	Push to GitHub
	git pull origin main	Pull from GitHub
UNDO	git restore <file>	Discard changes
	git reset HEAD~1	Undo last commit
	git stash	Stash changes
	git stash pop	Restore stash

DOCKER CHEAT SHEET

Category	Command	What It Does
IMAGES	<code>docker build -t myapp .</code>	Build image
	<code>docker build -t myapp:v1.0 .</code>	Build with version tag
	<code>docker images</code>	List images
	<code>docker rmi myapp</code>	Remove image
CONTAINERS	<code>docker run -p 3000:8081 myapp</code>	Run (your project)
	<code>docker run -d -p 3000:8081 myapp</code>	Run in background
	<code>docker run --name web myapp</code>	Run with name
	<code>docker ps</code>	Running containers
	<code>docker ps -a</code>	All containers
	<code>docker stop web</code>	Stop container
	<code>docker rm web</code>	Remove container
INSPECT	<code>docker logs web</code>	View logs
	<code>docker logs -f web</code>	Follow live logs
	<code>docker exec -it web sh</code>	Shell into container
CLEANUP	<code>docker system prune</code>	Remove all unused

YOUR PROJECT — QUICK COMMAND REFERENCE

Action	Command
Build Docker image	<code>docker build -t demo-nodejs .</code>
Run app (correct port mapping)	<code>docker run -p 3000:8081 demo-nodejs</code>
Run in background (detached)	<code>docker run -d -p 3000:8081 --name demo demo-nodejs</code>
View app in browser	<code>http://localhost:3000</code>
See app logs	<code>docker logs demo</code>
Stop running app	<code>docker stop demo</code>
Rebuild after code change	<code>docker stop demo && docker rm demo && docker build -t demo-nodejs . && docker run -d -p 3000:8081 --name demo demo-nodejs</code>
Push code to GitHub	<code>git add . && git commit -m 'your message' && git push origin main</code>
Create a feature branch	<code>git checkout -b feature/my-feature</code>
Merge feature branch to main	<code>git checkout main && git merge feature/my-feature</code>

WHAT GOES IN .gitignore

A .gitignore file tells Git which files to never track. Here's what your Node.js project should ignore:

```
node_modules/      # 3rd-party packages (re-installed via npm install)
.env               # Secrets, API keys — NEVER commit these!
.DS_Store          # Mac metadata files
*.log              # Log files
dist/              # Build output

# NOT ignored (you DO want to commit these):
# package.json      - dependency list
# package-lock.json - exact dependency versions
# Dockerfile        - container recipe
# app.js, *.html    - your actual code
```

BEGINNER TIPS — THINGS TO REMEMBER

Tip	Why
Commit often, push daily	Small commits are easier to understand and revert than giant ones.
Read the full error message	Docker and Git errors are descriptive. Read the FULL error, not just the first line.
Google the exact error	Copy the error message into Google. Stack Overflow and GitHub Issues are gold.
Use git status constantly	Before and after every git command. It's your compass.
Tag your Docker images	Use -t myapp:v1.0 so you can roll back to previous versions.
Keep main always working	Never push broken code to main. Use branches for experiments.
Fix your port mismatch	Change EXPOSE 3000 to EXPOSE 8081 in your Dockerfile — it currently mismatches app.js.

Keep building. Keep shipping.

Every expert was once a beginner who refused to give up.

Your next 5 steps:

1. Fix the port mismatch: EXPOSE 8081 in your Dockerfile
2. Add a .gitignore to your project
3. Create your first feature branch
4. Build and run your Docker container: `docker run -p 3000:8081 demo-nodejs`
5. Push your first Pull Request on GitHub

Muhammad Arham's DevOps Guide • demo_nodejs_webpage • 2026